UNIVERSITY OF UDINE
hic sunt futura

# Graph-Theoretical Arguments in support of a Quantum Declarative Manifesto

Alex Della Schiava, Carla Piazza, Riccardo Romanello
Department of Mathematics, Computer Science and Physics, University of Udine

Udine, June 21st, 2023 - *38th Italian Conference on Computational Logic*

# Graph Theory for Quantum Computing

Why "Graph-Theoretical Arguments?"

# Graph Theory for Quantum Computing

Why "Graph-Theoretical Arguments?" Quantum mechanics is weird.

# Graph Theory for Quantum Computing

Why "Graph-Theoretical Arguments?" Quantum mechanics is weird.

- "Oh yes, we also have probabilities but they are complex numbers."

# Graph Theory for Quantum Computing

Why "Graph-Theoretical Arguments?" Quantum mechanics is weird.

- "Oh yes, we also have probabilities but they are complex numbers."
- "Sure, entanglement is like a correlation but also isn't."

???

# Graph Theory for Quantum Computing

Why "Graph-Theoretical Arguments?" Quantum mechanics is weird.

- "Oh yes, we also have probabilities but they are complex numbers."
- "Sure, entanglement is like a correlation but also isn't."

???

Quantum Computing inherits all of this. Can we simplify it?

# Graph Theory for Quantum Computing

Why "Graph-Theoretical Arguments?" Quantum mechanics is weird.

- "Oh yes, we also have probabilities but they are complex numbers."
- "Sure, entanglement is like a correlation but also isn't."

???

Quantum Computing inherits all of this. Can we simplify it?

**Idea.** Characterize quantum computation structurally.

# Graph Theory for Quantum Computing

Why "Graph-Theoretical Arguments?" Quantum mechanics is weird.

- "Oh yes, we also have probabilities but they are complex numbers."
- "Sure, entanglement is like a correlation but also isn't."

???

Quantum Computing inherits all of this. Can we simplify it?

**Idea.** Characterize quantum computation structurally.

**How?** Through graphs!

# Quantum walks

Draw inspiration from classical probabilistic computation.

# Quantum walks

Draw inspiration from classical probabilistic computation.

- **Random walks:** Walker moves randomly.

# Quantum walks

Draw inspiration from classical probabilistic computation.

- **Random walks:** Walker moves randomly.
- **Quantum walks:** Walker moves unitarily.

# Quantum walks

Draw inspiration from classical probabilistic computation.

- **Random walks:** Walker moves randomly.
- **Quantum walks:** Walker moves unitarily.

---

Quantum walks on graphs are
**universal.**

# Quantum walks

Draw inspiration from classical probabilistic computation.

- **Random walks:** Walker moves randomly.
- **Quantum walks:** Walker moves unitarily.

---

Quantum walks on graphs are **universal.** $\Longrightarrow$ All quantum computations are graphs.

# Quantum walks

Draw inspiration from classical probabilistic computation.

- **Random walks:** Walker moves randomly.
- **Quantum walks:** Walker moves unitarily.

---

Quantum walks on graphs are **universal.** $\Longrightarrow$ All quantum computations are graphs.

Unitarity is **restrictive.** Quantum walks cannot be defined on any graph.

# Quantum walks

Draw inspiration from classical probabilistic computation.

- **Random walks:** Walker moves randomly.
- **Quantum walks:** Walker moves unitarily.

---

| | | |
|---|---|---|
| Quantum walks on graphs are **universal.** | $\Longrightarrow$ | All quantum computations are graphs. |
| Unitarity is **restrictive.** Quantum walks cannot be defined on any graph. | $\Longrightarrow$ | Not all graphs describe quantum computations. |

# Quantum walks

Draw inspiration from classical probabilistic computation.

- **Random walks:** Walker moves randomly.
- **Quantum walks:** Walker moves unitarily.

---

| | | |
|---|---|---|
| Quantum walks on graphs are **universal.** | $\Longrightarrow$ | All quantum computations are graphs. |
| Unitarity is **restrictive.** Quantum walks cannot be defined on any graph. | $\Longrightarrow$ | Not all graphs describe quantum computations. |

**Is there a programming paradigm that synergizes with this model?**

# Quantum walks

Draw inspiration from classical probabilistic computation.

- **Random walks:** Walker moves randomly.
- **Quantum walks:** Walker moves unitarily.

---

| | | |
|---|---|---|
| Quantum walks on graphs are **universal.** | $\Longrightarrow$ | All quantum computations are graphs. |
| Unitarity is **restrictive.** Quantum walks cannot be defined on any graph. | $\Longrightarrow$ | Not all graphs describe quantum computations. |

**Is there a programming paradigm that synergizes with this model?**

**Is declarative programming the answer?**

# Outline

1 Quantum walks

2 Graph-Theoretical Arguments

3 Declarative Programming for Quantum Computing

# Background (or: How I Learned to Stop Worrying and Write $|v\rangle$ instead of $\vec{v}$)

How different are quantum walks from random walks? Example on $n$ vertices:

# Background (or: How I Learned to Stop Worrying and Write $|v\rangle$ instead of $\vec{v}$)

How different are quantum walks from random walks? Example on $n$ vertices:

## Random walks

- State probability vector $\vec{q} \in [0,1]^n : (q_1, \cdots, q_n)$.
- Probability to be at vertex $i$: $q_i$.

# Background (or: How I Learned to Stop Worrying and Write $|v\rangle$ instead of $\vec{v}$)

How different are quantum walks from random walks? Example on $n$ vertices:

**Random walks**

- State probability vector $\vec{q} \in [0,1]^n : (q_1, \cdots, q_n)$.
- Probability to be at vertex $i$: $q_i$.

Stochastic $\mathbf{P}$

$$\begin{pmatrix} p_{1,1} & \cdots & p_{1,n} \\ \vdots & \ddots & \vdots \\ p_{n,1} & \cdots & p_{n,n} \end{pmatrix}$$

# Background (or: How I Learned to Stop Worrying and Write $|v\rangle$ instead of $\vec{v}$)

How different are quantum walks from random walks? Example on $n$ vertices:

**Random walks**

- State probability vector $\vec{q} \in [0,1]^n : (q_1, \cdots, q_n)$.
- Probability to be at vertex $i$: $q_i$.
- Rows of $\mathbf{P}$ sum to $1$.

Stochastic $\mathbf{P}$

$$\begin{pmatrix} p_{1,1} & \cdots & p_{1,n} \\ \vdots & \ddots & \vdots \\ p_{n,1} & \cdots & p_{n,n} \end{pmatrix} \begin{matrix} \sum = 1 \\ \vdots \\ \sum = 1 \end{matrix}$$

# Background (or: How I Learned to Stop Worrying and Write $|v\rangle$ instead of $\vec{v}$)

How different are quantum walks from random walks? Example on $n$ vertices:

## Random walks

- State probability vector $\vec{q} \in [0,1]^n : (q_1, \cdots, q_n)$.
- Probability to be at vertex $i$: $q_i$.
- Rows of $\mathbf{P}$ sum to $1$.

Stochastic $\mathbf{P}$

$$\begin{array}{cc} \begin{pmatrix} p_{1,1} & \cdots & p_{1,n} \\ \vdots & \ddots & \vdots \\ p_{n,1} & \cdots & p_{n,n} \end{pmatrix} & \begin{array}{c} \Sigma = 1 \\ \vdots \\ \Sigma = 1 \end{array} \end{array}$$

## Quantum walks

- Quantum state vector $|\psi\rangle \in \mathbb{C}^n$: $(c_1, \cdots, c_n)$.
- Probability to be at vertex $i$: $|c_i|^2$.

# Background (or: How I Learned to Stop Worrying and Write $|v\rangle$ instead of $\vec{v}$)

How different are quantum walks from random walks? Example on $n$ vertices:

## Random walks

- State probability vector $\vec{q} \in [0,1]^n : (q_1, \cdots, q_n)$.
- Probability to be at vertex $i$: $q_i$.
- Rows of $\mathbf{P}$ sum to $1$.

Stochastic $\mathbf{P}$

$$\begin{pmatrix} p_{1,1} & \cdots & p_{1,n} \\ \vdots & \ddots & \vdots \\ p_{n,1} & \cdots & p_{n,n} \end{pmatrix} \begin{matrix} \Sigma = 1 \\ \vdots \\ \Sigma = 1 \end{matrix}$$

## Quantum walks

- Quantum state vector $|\psi\rangle \in \mathbb{C}^n$: $(c_1, \cdots, c_n)$.
- Probability to be at vertex $i$: $|c_i|^2$.

Unitary $\mathbf{U}$

$$\begin{pmatrix} u_{1,1} & \cdots & u_{1,n} \\ \vdots & \ddots & \vdots \\ u_{n,1} & \cdots & u_{n,n} \end{pmatrix}$$

# Background (or: How I Learned to Stop Worrying and Write $|v\rangle$ instead of $\vec{v}$)

How different are quantum walks from random walks? Example on $n$ vertices:

## Random walks

- State probability vector $\vec{q} \in [0,1]^n : (q_1, \cdots, q_n)$.
- Probability to be at vertex $i$: $q_i$.
- Rows of $\mathbf{P}$ sum to $1$.

Stochastic $\mathbf{P}$

$$\begin{pmatrix} p_{1,1} & \cdots & p_{1,n} \\ \vdots & \ddots & \vdots \\ p_{n,1} & \cdots & p_{n,n} \end{pmatrix} \begin{matrix} \Sigma = 1 \\ \vdots \\ \Sigma = 1 \end{matrix}$$

## Quantum walks

- Quantum state vector $|\psi\rangle \in \mathbb{C}^n$: $(c_1, \cdots, c_n)$.
- Probability to be at vertex $i$: $|c_i|^2$.
- Rows and columns of $\mathbf{U}$ have length $1$.

Unitary $\mathbf{U}$

$$\begin{pmatrix} u_{1,1} & \cdots & u_{1,n} \\ \vdots & \ddots & \vdots \\ u_{n,1} & \cdots & u_{n,n} \end{pmatrix} \begin{matrix} \|\cdot\| = 1 \\ \vdots \\ \|\cdot\| = 1 \end{matrix}$$

$$\begin{matrix} \|\cdot\| = 1 & \cdots & \|\cdot\| = 1 \end{matrix}$$

# How restrictive is unitarity?

Unitary matrices are *more restrictive* than stochastic ones.

# How restrictive is unitarity?

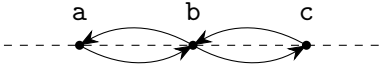
Unitary matrices are *more restrictive* than stochastic ones.

**How restrictive?**

# How restrictive is unitarity?

Unitary matrices are *more restrictive* than stochastic ones.

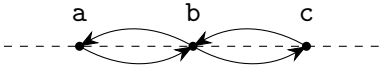
**How restrictive?** A lot: They do not support lines!



# How restrictive is unitarity?

Unitary matrices are *more restrictive* than stochastic ones.

**How restrictive?** A lot: They do not support lines!



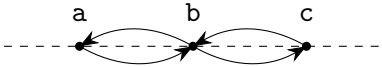

**Random walk**

$$\begin{array}{c} \mathtt{a} \\ \mathtt{b} \\ \mathtt{c} \end{array} \begin{pmatrix} 0 & 1 & 0 \\ 1/2 & 0 & 1/2 \\ 0 & 1 & 0 \end{pmatrix}$$

# How restrictive is unitarity?

Unitary matrices are *more restrictive* than stochastic ones.

**How restrictive?** A lot: They do not support lines!




**Random walk**

$$\begin{array}{c} \mathtt{a} \\ \mathtt{b} \\ \mathtt{c} \end{array} \begin{pmatrix} 0 & 1 & 0 \\ 1/2 & 0 & 1/2 \\ 0 & 1 & 0 \end{pmatrix}$$

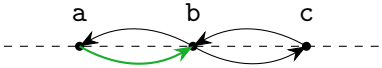
**Quantum walk**

$$\begin{array}{c} \mathtt{a} \\ \mathtt{b} \\ \mathtt{c} \end{array} \begin{pmatrix} 0 & u_{1,2} & 0 \\ u_{2,1} & 0 & u_{2,3} \\ 0 & u_{3,2} & 0 \end{pmatrix}$$

# How restrictive is unitarity?

Unitary matrices are *more restrictive* than stochastic ones.

**How restrictive?** A lot: They do not support lines!




**Random walk**

$$\begin{array}{c} a \\ b \\ c \end{array} \begin{pmatrix} 0 & 1 & 0 \\ 1/2 & 0 & 1/2 \\ 0 & 1 & 0 \end{pmatrix}$$

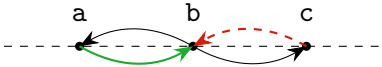
**Quantum walk**

$$\begin{array}{c} a \\ b \\ c \end{array} \begin{pmatrix} 0 & u_{1,2} & 0 \\ u_{2,1} & 0 & u_{2,3} \\ 0 & u_{3,2} & 0 \end{pmatrix}$$

# How restrictive is unitarity?

Unitary matrices are *more restrictive* than stochastic ones.

**How restrictive?** A lot: They do not support lines!




**Random walk**

$$\begin{array}{c} \mathtt{a} \\ \mathtt{b} \\ \mathtt{c} \end{array} \begin{pmatrix} 0 & 1 & 0 \\ 1/2 & 0 & 1/2 \\ 0 & 1 & 0 \end{pmatrix}$$

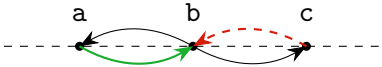
**Quantum walk**

$$\begin{array}{c} \mathtt{a} \\ \mathtt{b} \\ \mathtt{c} \end{array} \begin{pmatrix} 0 & 1 & 0 \\ u_{2,1} & 0 & u_{2,3} \\ 0 & u_{3,2} & 0 \end{pmatrix}$$

# How restrictive is unitarity?

Unitary matrices are *more restrictive* than stochastic ones.

**How restrictive?** A lot: They do not support lines!




**Random walk**

$$\begin{array}{c} \mathtt{a} \\ \mathtt{b} \\ \mathtt{c} \end{array} \begin{pmatrix} 0 & 1 & 0 \\ 1/2 & 0 & 1/2 \\ 0 & 1 & 0 \end{pmatrix}$$

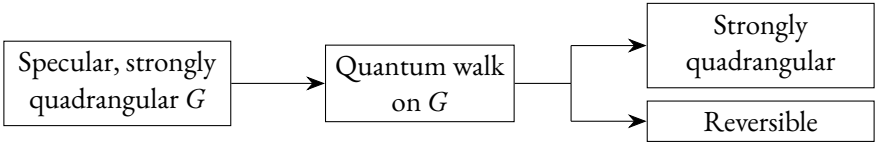
**Quantum walk**

$$\begin{array}{c} \mathtt{a} \\ \mathtt{b} \\ \mathtt{c} \end{array} \begin{pmatrix} 0 & 1 & 0 \\ u_{2,1} & 0 & u_{2,3} \\ 0 & 0 & 0 \end{pmatrix}$$

Edge $(\mathtt{c}, \mathtt{b})$ is lost!

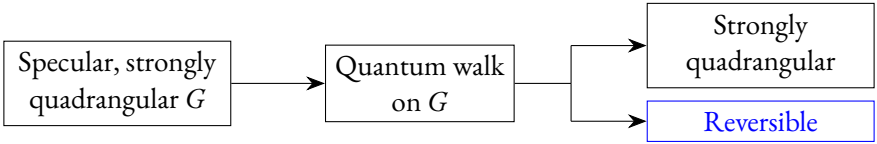
# Quantum walk amenable graphs

**Question 1.** Which graphs admit quantum walks? [3, 4, 5, 6]



# Quantum walk amenable graphs

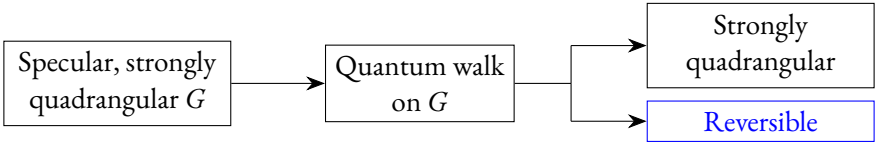
**Question 1.** Which graphs admit quantum walks? [3, 4, 5, 6]




## Reversibility

# Quantum walk amenable graphs

**Question 1.** Which graphs admit quantum walks? [3, 4, 5, 6]




## Reversibility

- Provides a connection to standard graph-theory.
- Almost equivalent to strong connectivity.

# Graph encoding in Quantum Computing

**Question 2.** How to encode general graphs into ones that admit quantum walks?

# Graph encoding in Quantum Computing

**Question 2.** How to encode general graphs into ones that admit quantum walks?

The paper reviews two encoding techniques:

1. Encoding general graphs into Eulerian ones [1].
2. Encoding irreversible graphs into reversible ones [3].

# Graph encoding in Quantum Computing

**Question 2.** How to encode general graphs into ones that admit quantum walks?

The paper reviews two encoding techniques:

1. Encoding general graphs into Eulerian ones [1].
2. Encoding irreversible graphs into reversible ones [3].

**Takeaway:** Irreversible graphs require classical operations to be encoded.

# Graph encoding in Quantum Computing

**Question 2.** How to encode general graphs into ones that admit quantum walks?

The paper reviews two encoding techniques:

1. Encoding general graphs into Eulerian ones [1].
2. Encoding irreversible graphs into reversible ones [3].

**Takeaway:** Irreversible graphs require classical operations to be encoded.

$$\text{Reversibility} \implies \text{Only unitarity is needed} \Downarrow \textbf{Quantum walks}$$

# How does this help us?

Assuming to have actual quantum supremacy in the future...

# How does this help us?

Assuming to have actual quantum supremacy in the future...

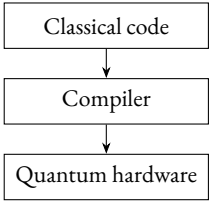
...will there be need for quantum software engineers?

# How does this help us?

Assuming to have actual quantum supremacy in the future…

…will there be need for quantum software engineers?

**Idea.** Keep classical code. Delegate "quantum mechanical duty" to the **compiler.**

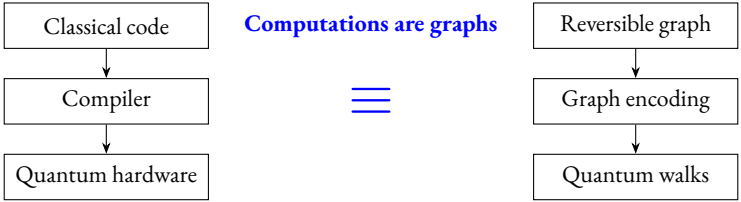


# How does this help us?

Assuming to have actual quantum supremacy in the future…

…will there be need for quantum software engineers?

**Idea.** Keep classical code. Delegate "quantum mechanical duty" to the **compiler.**

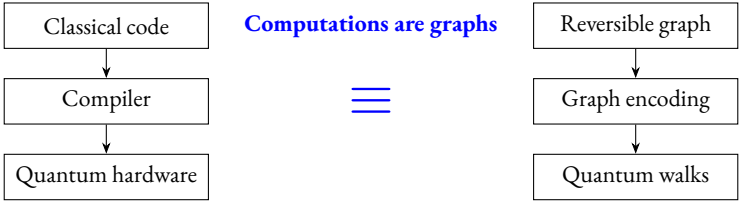


# How does this help us?

Assuming to have actual quantum supremacy in the future…

…will there be need for quantum software engineers?

**Idea.** Keep classical code. Delegate "quantum mechanical duty" to the **compiler.**




**Which programming paradigm is more suitable for this model?**

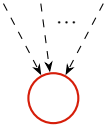
# Why not imperative programming?

**Hint 1.** Imperative programs admit graph representations...

# Why not imperative programming?

**Hint 1.** Imperative programs admit graph representations...

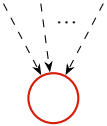
- ...which are never reversible: termination states = sink vertices.



# Why not imperative programming?

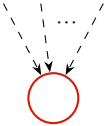
**Hint 1.** Imperative programs admit graph representations...

- ...which are never reversible: termination states = sink vertices.




**Hint 2.** What about reversibile computation?

# Why not imperative programming?



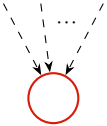

**Hint 1.** Imperative programs admit graph representations...

- ...which are never reversible: termination states = sink vertices.

**Hint 2.** What about reversibile computation?

- Any boolean function may be embedded into one that is reversible.

# Why not imperative programming?




**Hint 1.** Imperative programs admit graph representations...

- ...which are never reversible: termination states = sink vertices.

**Hint 2.** What about reversibile computation?

- Any boolean function may be embedded into one that is reversible.
- Extra qubits required: something current quantum computers cannot offer.

# Why declarative programming?

**Hint 1.** No-cloning theorem. $\Longrightarrow$ No assignments in Quantum Computing.

# Why declarative programming?

**Hint 1.** No-cloning theorem. $\Longrightarrow$ No assignments in Quantum Computing.

---

## Functional programming

# Why declarative programming?

**Hint 1.** No-cloning theorem. $\Longrightarrow$ No assignments in Quantum Computing.

## Functional programming

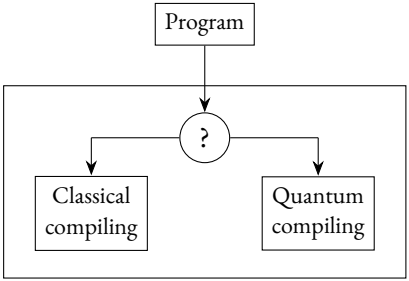
- Any function can be made reversible.
- Is it always worth it?

# Why declarative programming?

**Hint 1.** No-cloning theorem. $\Longrightarrow$ No assignments in Quantum Computing.

## Functional programming

- Any function can be made reversible.
- Is it always worth it?
- **Hint 2.** Exploit compositionality of denotational semantics.



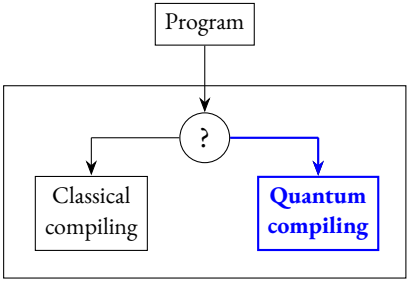
# Why declarative programming?

**Hint 1.** No-cloning theorem. $\Longrightarrow$ No assignments in Quantum Computing.

## Functional programming

- Any function can be made reversible.
- Is it always worth it?
- **Hint 2.** Exploit compositionality of denotational semantics.

Want to compute the DFT?



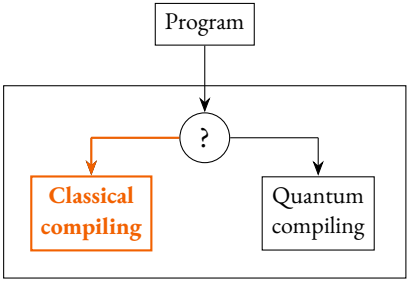
# Why declarative programming?

**Hint 1.** No-cloning theorem. $\Longrightarrow$ No assignments in Quantum Computing.

## Functional programming

- Any function can be made reversible.
- Is it always worth it?
- **Hint 2.** Exploit compositionality of denotational semantics.

Want to sort an array?



# Why declarative programming? (cont'd)

## Logic programming

# Why declarative programming? (cont'd)

## Logic programming

**Hint 3.** Stable model search can be reduced to Grover's search [2].

**Grover's search**

Unstructured search over $N$ elements.
Solvable in $\mathcal{O}(\sqrt{N})$ time on a quantum computer.

# Why declarative programming? (cont'd)

## Logic programming

**Hint 3.** Stable model search can be reduced to Grover's search [2].

- Any ASP program can be executed on quantum architecture.

**Grover's search**

Unstructured search over $N$ elements.
Solvable in $\mathcal{O}(\sqrt{N})$ time on a quantum computer.

# Why declarative programming? (cont'd)

## Logic programming

**Hint 3.** Stable model search can be reduced to Grover's search [2].

- Any ASP program can be executed on quantum architecture.
- **Idea.** What if we formalized the same reduction from SAT?

**Grover's search**

Unstructured search over $N$ elements.
Solvable in $\mathcal{O}(\sqrt{N})$ time on a quantum computer.

# Why declarative programming? (cont'd)

## Logic programming

**Hint 3.** Stable model search can be reduced to Grover's search [2].

- Any ASP program can be executed on quantum architecture.
- **Idea.** What if we formalized the same reduction from SAT?

$\Downarrow$

**Grover's search**

Unstructured search over $N$ elements.
Solvable in $\mathcal{O}(\sqrt{N})$ time on a quantum computer.

**SAT solving-based programs would be executable on quantum architecture!**

# Efficient storing of graphs

Unitary matrices allow to efficiently store graphs on quantum hardware.

How would this apply to ASP and Minizinc?

# Efficient storing of graphs

Unitary matrices allow to efficiently store graphs on quantum hardware.

How would this apply to ASP and Minizinc?

## Dependency graphs in ASP

- Tight programs are troubling: acyclic dependency graphs.
- Reversible graphs are characterized by loops.

# Efficient storing of graphs

Unitary matrices allow to efficiently store graphs on quantum hardware.

How would this apply to ASP and Minizinc?

## Dependency graphs in ASP

- Tight programs are troubling: acyclic dependency graphs.
- Reversible graphs are characterized by loops.

Not a promising start. But…

# Efficient storing of graphs

Unitary matrices allow to efficiently store graphs on quantum hardware.

How would this apply to ASP and Minizinc?

## Dependency graphs in ASP

- Tight programs are troubling: acyclic dependency graphs.
- Reversible graphs are characterized by loops.

Not a promising start. But…

These are graphs, not computations. $\Rightarrow$ More liberal encoding approach!

# Conclusions

Quantum walks provide insights on what a quantum computation is...

# Conclusions

Quantum walks provide insights on what a quantum computation is...

...and what it cannot be.

# Conclusions

Quantum walks provide insights on what a quantum computation is...

...and what it cannot be.

These insights outline an ideal candidate for compilation to quantum architectures:

Declarative programming

# Conclusions

Quantum walks provide insights on what a quantum computation is...

...and what it cannot be.

These insights outline an ideal candidate for compilation to quantum architectures:

Declarative programming

(As for ASP, there already is rigorous evidence!)

# Conclusions

Quantum walks provide insights on what a quantum computation is...

...and what it cannot be.

These insights outline an ideal candidate for compilation to quantum architectures:

Declarative programming

(As for ASP, there already is rigorous evidence!)

Still a lot of work to be done. Is there need to learn Quantum Computing for it?

# Conclusions

Quantum walks provide insights on what a quantum computation is...

...and what it cannot be.

These insights outline an ideal candidate for compilation to quantum architectures:

Declarative programming

(As for ASP, there already is rigorous evidence!)

Still a lot of work to be done. Is there need to learn Quantum Computing for it?

No, the common language of Graph Theory will do!

# Reversible vs strongly connected graphs

### Strong connectivity

For any two vertices $u, v$ there exists a path from $u$ to $v$.

# Reversible vs strongly connected graphs

### Strong connectivity

For any two vertices $u, v$ there exists a path from $u$ to $v$.

### Reversibility

For any edge $(u, v)$ there exists a path from $v$ to $u$.

Any strongly connected graph is reversible.

# Reversible vs strongly connected graphs

## Strong connectivity

For any two vertices $u, v$ there exists a path from $u$ to $v$.

## Reversibility

For any edge $(u, v)$ there exists a path from $v$ to $u$.

Any strongly connected graph is reversible.

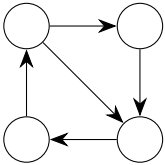


Fig.: Strongly connected graph.

# Reversible vs strongly connected graphs

### Strong connectivity

For any two vertices $u, v$ there exists a path from $u$ to $v$.

### Reversibility

For any edge $(u, v)$ there exists a path from $v$ to $u$.

Any strongly connected graph is reversible.

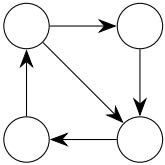


Fig.: Strongly connected graph.

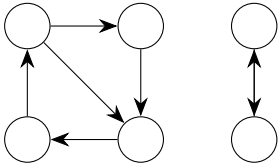


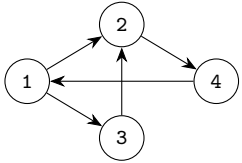
Fig.: Reversible non-strongly connected graph

# Decoding quadrangularity

**Quadrangularity.** Any two vertices *never* share *exactly one* in- or out-neighbour.

# Decoding quadrangularity

**Quadrangularity.** Any two vertices *never* share *exactly one* in- or out-neighbour.

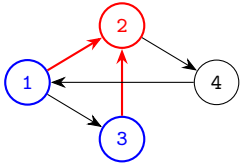



$$\begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

Fig.: Non-quadrangular graph and its adjacency matrix.

# Decoding quadrangularity

**Quadrangularity.** Any two vertices *never* share *exactly one* in- or out-neighbour.

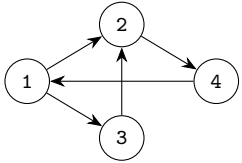



$$\begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

Fig.: Non-quadrangular graph and its adjacency matrix.

# Decoding quadrangularity

**Quadrangularity.** Any two vertices *never* share *exactly one* in- or out-neighbour.




$$\begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

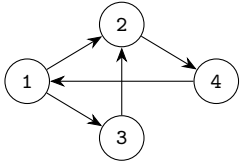
Fig.: Non-quadrangular graph and its adjacency matrix.

## Strong quadrangularity

Let $S$ be any set of $k$ vertices, where all $u \in S$ share at least one in- (out-) neighbour with some other $v \in S$.

# Decoding quadrangularity

**Quadrangularity.** Any two vertices *never* share *exactly one* in- or out-neighbour.




$$\begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

Fig.: Non-quadrangular graph and its adjacency matrix.

## Strong quadrangularity

Let $S$ be any set of $k$ vertices, where all $u \in S$ share at least one in- (out-) neighbour with some other $v \in S$. $\Rightarrow$ The union of the pairwise shared in- (out-) neighbours by the vertices in $S$ must be at least $k$.

# Decoding quadrangularity cont'd

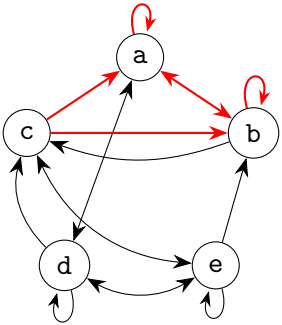
- Quadrangular graph not strongly quadrangular.

# Decoding quadrangularity cont'd

- Quadrangular graph not strongly quadrangular.
- The 3 vertices $\{\mathtt{a},\mathtt{b},\mathtt{c}\}$ share 2 out-neighbours:

$$\{\mathtt{a},\mathtt{b}\}.$$

- $2 < 3$: strong quadrangularity is violated.



# Bibliography I


[1] D. Della Giustina et al. "Directed Graph Encoding in Quantum Computing Supporting Edge-Failures". In: *Reversible Computation*. Ed. by Claudio Antares Mezzina and Krzysztof Podlaski. Cham: Springer International Publishing, 2022, pp. 75–92.

[2] David Meyer, James Pommersheim, and Jeffrey Remmel. "Finding stable models via quantum computation.". In: Jan. 2004, pp. 285–291.

[3] A. Montanaro. "Quantum Walks on Directed Graphs". In: *Quantum Info. Comput.* 7.1 (2007), pp. 93–102.

[4] S. Severini. *Graphs of unitary matrices*. 2003. DOI: 10.48550/ARXIV.MATH/0303084.

# Bibliography II

[5] S. Severini. "On the Digraph of a Unitary Matrix". In: *SIAM Journal on Matrix Analysis and Applications* 25.1 (2003), pp. 295–300. DOI: 10.1137/s0895479802410293.

[6] S. Severini. *The underlying digraph of a coined quantum random walk*. 2002. DOI: 10.48550/ARXIV.QUANT-PH/0210055.